

4. Thực hành

Phase 0: Healthy Baseline (0–10 phút)

Mục tiêu: Hiểu hệ thống đang chạy thế nào

```
make reset  
make baseline  
pytest tests_public -q
```

Trả lời các câu hỏi sau:

1. Dataset nào là critical? (Impact trực tiếp đến business)
2. Downstream consumer nào phụ thuộc vào dataset đó?
3. Metric nào cho biết data không đáng tin?

Xem lineage graph:

```
cat data/baseline/lineage_graph.json
```

Phase 1: Contract + Validation (10–30 phút)

Files cần làm việc:

- [contracts/orders_contract.yaml](#)
- [src/contract_validator.py](#)
- [gx/validate_orders.py](#)

Bài tập bắt buộc:

1. Thêm **type validation** trong `src/contract_validator.py` :

- Kiểm tra `order_id` là integer
- Kiểm tra `amount` là number
- Kiểm tra `created_at` là datetime

2. Thêm **freshness validation**:

- Đọc `freshness` từ contract YAML
- So sánh `updated_at` với `max_delay_minutes`
- Phân severity: `critical` , `warning` , `info`

3. Phân loại **action**:

- `block` : Dừng pipeline, ngăn data đi tiếp
- `quarantine` : Tách data lỗi sang side table
- `warn` : Ghi log, tiếp tục

4. Test với **fault**:

```
python scripts/inject_fault.py duplicate_pk  
make baseline
```

Kiểm tra contract validator có bắt được duplicate key không.

Bài tập Advanced (bonus):

- Build Great Expectations Expectation Suite + ValidationDefinition + Checkpoint
- Implement automatic quarantine khi contract fail ở severity `critical`

Phase 2: dbt Transformation Protection (30-50 phút)

Files cần làm việc:

- `dbt_project/models/marts/`
- `dbt_project/models/staging/`

```
make reset  
make dbt
```

Bài tập bắt buộc:

1. Thêm generic data tests (ít nhất 2):

- Test trong file `models/marts/schema.yml` hoặc `models/staging/schema.yml`
- Ví dụ: `not_null` , `unique` , `accepted_values` , `relationship`

2. Thêm singular business test:

- File riêng trong `models/marts/singular_tests/`

3. Giải thích:

- Tại sao `not_null` / `unique` không phải dbt **unit test**?
- Unit test dùng cho logic transformation, không phải data quality

Bài tập Strong Challenge (bonus):

- Viết unit test nhỏ nhất để expose revenue inflation khi customer dimension có nhiều active versions (SCD issue)

Phase 3: Anomaly Detection (50-70 phút)

Files cần làm việc:

- [observability/anomaly.py](#)
- [observability/distribution.py](#)

```
make reset  
python scripts/inject_fault.py volume_drop  
make baseline
```

Bài tập bắt buộc:

1. Baseline detector đã bắt được volume drop:

- Chạy Z-score hoặc baseline đơn giản

- Verify `is_anomaly = True`

2. Giải thích khi nào Z-score sai:

- Khi nào `std = 0`?
- Khi nào có seasonality?
- Khi nào outlier ảnh hưởng?

Bài tập Advanced (bonus +3 điểm):

Nâng cấp `method="auto"` trong `detect_anomaly()` :

```
# Cải thiện auto mode để xử lý:  
# - same-weekday baseline (Weekend vs Weekday có traffic khác nhau)  
# - median/MAD thay vì mean/std  
# - rolling baseline  
# - EWMA
```

Phase 4: Lineage & Blast Radius (70-85 phút)

Files cần làm việc:

- [observability/lineage.py](#)
- [data/baseline/lineage_graph.json](#)

Bài tập bắt buộc:

Trace downstream impact bằng code:

```
from observability.lineage import load_graph, get_downstream_assets  
  
graph = load_graph("data/baseline/lineage_graph.json")  
affected = get_downstream_assets(graph, "stg_orders")  
print(f"stg_orders lỗi -> impact: {affected}")
```

Output mong đợi:

```
['fct_daily_revenue', 'ceo_revenue_dashboard']
```

Bài tập Advanced (bonus +7 điểm):

- Parse `dbt_project/target/manifest.json` sau `dbt build`
 - Implement column-level transitive lineage
 - Optional: emit OpenLineage events
-

Phase 5: SLO/Error Budget (85–100 phút)

Files cần làm việc:

- [observability/slo.py](#)

Bài tập bắt buộc:

Tính toán SLO cho trường hợp sau:

Cho SLO = 99.5%, 2 bad checks/100 checks:

- actual bad rate?
- allowed bad rate?
- burn rate?
- breached?

```
from student_api import slo_status

result = slo_status(target=0.995, bad_events=2, total_events=100)
print(result)
```

Bài tập Advanced (bonus +7 điểm):

Implement `multiwindow_burn()` :

- Transient spike ngắn → không page
 - Sustained fast burn → page ngay
-

Phase 6: Mystery Incident (100–115 phút)

Lưu ý: Giảng viên sẽ đưa fault folder riêng hoặc incoming dataset khác.

KHÔNG ĐƯỢC xem script tạo fault.

Chỉ dùng evidence từ:

- Contracts/validation
- dbt tests
- Anomaly metrics
- Lineage
- SLO
- Raw-data exploration có lý do

Trả lời đầy đủ:

1. What happened?
2. When did it start?
3. Root cause?
4. Blast radius?
5. Mitigation?
6. Recovery verification?
7. Prevention?

Phase 7: Report (115-120 phút)

Hoàn thiện các files:

1. [reports/incident_report.md](#)
2. [reports/agent_log.md](#)

[← Quay lại](#)

4 / 7

[Tiếp](#)